Homework 2: Collision check and RRT

16-662: Robot Autonomy

Spring 2026

Due: Tuesday, Feb 24, 2026 at 11:59 PM ET

Instructor: Oliver Kroemer

Teaching Assistants: Sreeharsha Paruchuri, Abhinandan Vellanki

Introduction

This homework will focus on **collision avoidance and motion planning** discussed in lecture. You will first implement a collision detection algorithm, and then use it to determine if two cuboids are intersecting. You will then implement a **Rapidly-exploring Random Tree (RRT)** and a **Probabilistic Roadmap (PRM)**, as discussed in the lectures, and use it to plan a collision-free path from a start to target joint configuration. Results from the motion planning algorithm will be visualized in the MuJoCo simulation environment.

AI Usage Policy

The use of AI to generate code for this homework is **strictly forbidden**. You are instead recommended to attend the Teaching Assistants' office hours or research by yourself or with your peers.

Part 1: MuJoCo Set Up (using venv)

Note: You can use the conda environment you created in Assignment 1 if preferred. If so, you may skip this part.

Creating a Python `venv`

To set up MuJoCo:

1. Navigate to the home directory.
2. Create a Python virtual environment by running:

   ```
   python3 -m venv ./<your_env_name>
   ```

   Replace `<your_env_name>` with a name of your choice.

3. Activate the virtual environment by running the activation script in the `bin` folder:

   ```
   source ./<your_env_name>/bin/activate
   ```

4. Within this environment, run:

   ```
   which python
   ```

   Ensure that the output points to a location inside the virtual environment path as a sanity check.

5. Install the required dependencies using `pip`:

   (a) `python -m pip install mujoco`

   (b) `python -m pip install quaternion`

   (c) `python -m pip install matplotlib`

6. You can now use the packages from the Python virtual environment to run the homework files. Make sure to activate the virtual environment whenever you run the files.

   IMPORTANT: Ensure there are no whitespaces in your folder names as this can cause problems while running the MuJoCo executable, especially when working inside Python environments.

   MacOS: To run Python scripts that use MuJoCo, use the mjpython binary to run them, i.e., replace the usual `python script.py` with `mjpython script.py`

Part 2: Cuboid-Cuboid Collision Detection

In the first part of the homework, we will implement a collision detection algorithm for use in the remainder of the assignment. Collision avoidance is a critical aspect of robot motion planning, as we want our robots to operate safely around objects, other robots, and humans.

A first-order approach for collision avoidance is to define cuboids around objects (also known as rectangular prisms) and detect whether these cuboids intersect. In this context, these cuboids are also referred to as *bounding boxes*. For our collision detection algorithm, we recommend implementing the *Separating Axis Theorem* to determine when two cuboids intersect. We will only use cuboids to parameterize collision geometry in this assignment.

For this assignment, we will parameterize a cuboid through the specification of:

1. The pose of its centroid, which serves as the cuboid local frame; and
2. The specification of the cuboid's lengths in the local x, y, and z dimensions.

We use the standard roll–pitch–yaw convention for orientation, where the (r, p, y) tuple represents the angle about the body's x-axis (roll), y-axis (pitch), and z-axis (yaw), respectively. The function `BlockDesc2Points(H, Dim)` provided in `RobotUtil.py` takes as input an object pose (specified by a homogeneous transformation matrix H) and object dimensions (length (x), width (y), and height (z)), and returns the bounding box of the object along with the axes given by the rotation matrix.

2.1 Implement Cuboid–Cuboid Collision Checker

To complete this part of the homework, please implement the *Separating Axis Theorem*. More specifically, implement the functions `CheckPointOverlap()` and `CheckBoxBoxCollision()` in `RobotUtil.py`.

Evaluating cuboid–cuboid collision checker: To evaluate your collision checker, for the cuboids listed in Table 1, provide a yes/no response indicating whether the test case cuboid is colliding with the reference cuboid. The reference cuboid is defined as follows (with centroid pose coincident with the world frame):

Origin (m) (x, y, z)**:**	$(0, 0, 0)$
Orientation (rad) (r, p, y)**:**	$(0, 0, 0)$
Dimensions (m) (d_x, d_y, d_z)**:**	$(3, 1, 2)$

Consider cases where cuboid faces are touching in the same plane to be a collision, as well as cases where one cuboid completely encloses another cuboid.

You can then run the `RobotUtil.py` file, which contains test cases implemented in the `main` function.

Test Case	**Origin (m)** (x, y, z)	**Orientation (rad)** (r, p, y)	**Dimensions (m)** (d_x, d_y, d_z)
1	$(0, 1, 0)$	$(0, 0, 0)$	$(0.8, 0.8, 0.8)$
2	$(1.5, -1.5, 0)$	$(1, 0, 1.5)$	$(1, 3, 3)$
3	$(0, 0, -1)$	$(0, 0, 0)$	$(2, 3, 1)$
4	$(3, 0, 0)$	$(0, 0, 0)$	$(3, 1, 1)$
5	$(-1, 0, -2)$	$(0.5, 0, 0.4)$	$(2, 0.7, 2)$
6	$(1.8, 0.5, 1.5)$	$(-0.2, 0.5, 0)$	$(1, 3, 1)$
7	$(0, -1.2, 0.4)$	$(0, 0.785, 0.785)$	$(1, 1, 1)$
8	$(-0.8, 0, -0.5)$	$(0, 0, 0.2)$	$(1, 0.5, 0.5)$

Table 1: Cuboid test cases for evaluating cuboid–cuboid collision checking.

2.2 Implement Robot Bounding Boxes

The `Franka.py` file has been expanded to include collision checking based on the `RobotUtil.py` functions that you have implemented. The constructor also includes additional parameters defining

the arm's bounding boxes. We provide the bounding boxes for each of the links of the Franka arm in `Franka.py`.

The variable `self.Cdesc` contains the poses (roll, pitch, yaw, and $x/y/z$) of each robot arm bounding box, which you will use to define the transforms for each link. Use these bounding box descriptions along with the associated link-to-forward transforms to compute the bounding box for each corresponding robot link.

The computation of the bounding boxes for a given joint configuration is not provided. Complete the missing code in the function `CompCollisionBlockPoints()`.

You can then run the `Franka.py` file, which should plot the collision boxes for the Franka robot.

2.3 Submission

For the collision detection section, submit a PDF page containing the following items:

- A list of yes/no responses indicating whether the cuboids in Table 1 are colliding with the reference cuboid.
- A picture of the robot's bounding boxes when it is in its home configuration (all joint angles set to zero). Use the `PlotCollisionBlockPoints` function in `Franka.py`.

Part 3: Franka RRT Motion Planning

3.1 Implementation

In this section of the homework, you will implement a Rapidly-exploring Random Tree (RRT) motion planner in joint space. This planner should provide a feasible, collision-free path from an initial joint configuration to a given target joint configuration.

Joint Name	**Initial**	**Goal**
joint_1	$-90°$	$90°$
joint_2	$-90°$	$-90°$
joint_3	$90°$	$-90°$
joint_4	$-90°$	$-90°$
joint_5	$0°$	$0°$
joint_6	$150°$	$150°$
joint_7	$0°$	$0°$

The motion planner should plan in joint space for joints 1 through 5. We are not concerned with moving the gripper fingers in this assignment, so planning for joint 6 and joint 7 is not necessary.

You will use your cuboid collision detection code from the previous section of this homework to perform collision detection. This includes collisions with obstacles in the scene as well as self-collisions with the robot's base.

We have provided the bounding boxes for the obstacles in the scene, the robot base, and the camera mount in the `RRTQuery.py` file. The robot base and camera mount should be treated as obstacles in the environment, since the robot base will remain fixed for this assignment.

Implement RRT. You should implement your RRT planner in `RRTQuery.py` to find a collision-free path. This code provides bounding boxes for all obstacles present in the environment (`pointObs` and `envaxes`). As part of your implementation, it is recommended to expand the tree using the RRT-Connect approach and to incorporate a goal bias to guide tree growth. The `Franka.py` class includes a function `SampleRobotConfig` for sampling configurations within the joint limits.

Visualizing the RRT Solution. To visualize the solution, run the `RRTQuery.py` file. If a valid plan has been found, you should see the robot execute the planned trajectory in MuJoCo.

Implement Path Shortening The path found by the RRT planner may be a bit winding and unnecessarily long. In the `RRTQuery.py` file, once a plan has been found, implement path shortening. Remember to check the collisions along any shortened paths that you introduce.

3.2 Submission

For the RRT section, submit the following items:

- A video of the Franka robot executing a plan produced by the RRT *without* path shortening.
- A video of the Franka robot executing a plan produced by the RRT *with* path shortening.

Please upload the videos as *unlisted* on YouTube and include the corresponding links in the report.

To Submit

- A PDF containing the submission details described in Sections 3.2 and 4.2.
- Your code, submitted as a single zipped file.

That's it! You have completed HW 2! =D